1. Imports

In [1]:
```
from bs4 import BeautifulSoup
```

2. Create a `soup` object

- It's the basic building block of the beautiful soup library, and all our future work happens on this object itself.

In [2]:
```
with open("html-doc.html") as file:
    soup = BeautifulSoup(file, "html.parser")
```

In [3]:
```
type(soup)
```

Out[3]:
```
bs4.BeautifulSoup
```

3. Basics of `soup` object

- `prettify()`
- individual tags:
  - `title`
  - `a`
  - `p`
- `text`
- `name`
- `parent`
- `children`
- `descendants`
- `get_text()`
- `find()`
- `find_all()`
- `get()` / square bracket notation

In [4]:
```
# prettify

soup.prettify()
```

Out[4]:
```
'<html>\n <head>\n  <title>\n   The Dormouse\'s story\n  </title>\n </head>\n <body>\n  <p class="title">\n   <b>\n    The Dormouse\'s story\n   </b>\n  </p>\n  <p class="story">\n   Once upon a time there were three little sisters; and their names were\n   <a class="sister" href="http://example.com/elsie" id="link1">\n    Elsie\n   </a>\n   ,\n   <a class="sister" href="http://example.com/lacie" id="link2">\n    Lacie\n   </a>\n   and\n   <a class="sister" href="http://example.com/tillie" id="link3">\n    Tillie\n   </a>\n   ;\nand they lived at the bottom of a well.\n  </p>\n  <p class="story">\n   ...\n  </p>\n </body>\n</html>\n'
```

In [5]:
```
type(soup.prettify())
```

Out[5]:
```
str
```

In [6]:
```
print(soup.prettify())
```

```
<html>
 <head>
  <title>
   The Dormouse's story
  </title>
 </head>
 <body>
  <p class="title">
   <b>
    The Dormouse's story
   </b>
  </p>
  <p class="story">
   Once upon a time there were three little sisters; and their names were
   <a class="sister" href="http://example.com/elsie" id="link1">
    Elsie
   </a>
   ,
   <a class="sister" href="http://example.com/lacie" id="link2">
    Lacie
   </a>
   and
   <a class="sister" href="http://example.com/tillie" id="link3">
    Tillie
   </a>
   ;
and they lived at the bottom of a well.
  </p>
  <p class="story">
   ...
  </p>
 </body>
</html>
```

- Individual tags:
- example: `title` , `a` , `p`
- when there are multiple individual tags in html document
- It always gives the first occurence of tag in html document

In [7]:
```
soup.title
```

Out[7]: <title>The Dormouse's story</title>

In [8]:
```
soup.a
```

Out[8]: <a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>

In [9]:
```
soup.p
```

Out[9]: The Dormouse's story

In [ ]:
```
# text: It gives the content that is enclosed within a tag.
```

In [10]:
```
soup.title
```

Out[10]: <title>The Dormouse's story</title>

In [11]:
```
type(soup.title)
```

Out[11]: bs4.element.Tag

In [12]:
```
soup.title.text
```

Out[12]: "The Dormouse's story"

In [13]:
```
type(soup.title.text)
```

Out[13]: str

In [14]:
```
soup.title.text.upper()
```

Out[14]: "THE DORMOUSE'S STORY"

In [15]:
```
soup.title.text.lower()
```

Out[15]: "the dormouse's story"

In [16]:
```
soup.title.text.split()
```

Out[16]: ['The', "Dormouse's", 'story']

In [ ]:
```
# name: It gives tag name
```

In [17]:
```
soup.title
```

Out[17]: <title>The Dormouse's story</title>

In [18]:
```
soup.title.name
```

Out[18]: 'title'

In [19]:
```
type(soup.title.name)
```

Out[19]: str

In [ ]:
```
# parent: It gives the parent tag that surrounds the current working tag.
```

In [20]:
```
soup.title.parent
```

Out[20]:
```
<head><title>The Dormouse's story</title></head>
```

In [21]:
```
soup.title.parent.parent
```

Out[21]:
```
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their na
mes were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p></body></html>
```

In [ ]:
```
# children: It gives the immediate child tags of the current working tag
```

In [22]:
```
soup.body
```

Out[22]:
```
<body>
<p class="title"><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their na
mes were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p></body>
```

In [23]:
```
soup.body.children
```

Out[23]:
```
<generator object Tag.children.<locals>.<genexpr> at 0x000001AA4C6F1700>
```

In [24]:
```
for child in soup.body.children:
    print(child)
```

```
<p class="title"><b>The Dormouse's story</b></p>


<p class="story">Once upon a time there were three little sisters; and their name
s were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>


<p class="story">...</p>
```

In [25]:
```
soup.head
```

Out[25]:
```
<head><title>The Dormouse's story</title></head>
```

In [26]:
```
soup.head.children
```

Out[26]: <generator object Tag.children.<locals>.<genexpr> at 0x000001AA4C6F1A40>

In [27]:
```
for child in soup.head.children:
    print(child)
```

```
<title>The Dormouse's story</title>
```

In [ ]:
```
# descendants: It provides the immediate child tags and their nested child tags
```

In [28]:
```
soup.body
```

Out[28]:
```
<body>
<p class="title">><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their na
mes were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p></body>
```

In [29]:
```
soup.body.descendants
```

Out[29]: <generator object Tag.descendants at 0x000001AA4C5E2350>

In [30]:
```
for descendant in soup.body.descendants:
    print(descendant)
```

```
<p class="title">><b>The Dormouse's story</b></p>
<b>The Dormouse's story</b>
The Dormouse's story


<p class="story">Once upon a time there were three little sisters; and their name
s were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
Once upon a time there were three little sisters; and their names were

<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>
Elsie
,

<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>
Lacie
 and

<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
Tillie
;
and they lived at the bottom of a well.


<p class="story">...</p>
...
```

- you make sure to compare the results of `children` and `descendant`

In [ ]:
```
# get_text(): It ignores the tags and gives the text of all the tags as a result
```

In [31]:
```
soup.body
```

Out[31]:
```
<body>
<p class="title"><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their na
mes were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p></body>
```

In [32]:
```
soup.body.get_text()
```

Out[32]:
```
"\nThe Dormouse's story\nOnce upon a time there were three little sisters; and
their names were\nElsie,\nLacie and\nTillie;\nand they lived at the bottom of a
well.\n..."
```

In [33]:
```
type(soup.body.get_text())
```

Out[33]:
```
str
```

In [34]:
```
print(soup.body.get_text())
```

```
The Dormouse's story
Once upon a time there were three little sisters; and their names were
Elsie,
Lacie and
Tillie;
and they lived at the bottom of a well.
...
```

In [35]:
```
print(soup.body.get_text().strip())
```

```
The Dormouse's story
Once upon a time there were three little sisters; and their names were
Elsie,
Lacie and
Tillie;
and they lived at the bottom of a well.
...
```

In [36]:
```
soup.title
```

Out[36]:
```
<title>The Dormouse's story</title>
```

In [37]:
```
soup.title.text
```

Out[37]:
```
"The Dormouse's story"
```

In [38]:
```
soup.title.get_text()
```

Out[38]:
```
"The Dormouse's story"
```

In [ ]:
```
# find()

"""
The find method can be a bit complex, and we'll explore its intricacies in the n
It's important to understand what HTML content is.
In an HTML tag, you can define a class or an ID to make it easier to distinguish
When you need to access a specific tag amidst this complexity, the find method p
It includes additional parameters that help facilitate precise access, which we
"""
```

In [38]:
```
soup.body
```

Out[38]:
```
<body>
<p class="title"><b>The Dormouse's story</b></p>
<p class="story">Once upon a time there were three little sisters; and their na
mes were
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a> and
<a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>
<p class="story">...</p></body>
```

In [39]:
```
soup.a
```

Out[39]:
```
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>
```

In [40]:
```
soup.find('a')
```

Out[40]:
```
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>
```

In [ ]:
```
# find_all()
```

In [41]:
```
soup.find_all('a')
```

Out[41]:
```
[<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
 <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
 <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]
```

In [43]:
```
type(soup.find_all('a'))
```

Out[43]:
```
bs4.element.ResultSet
```

In [ ]:
```
# get()
```

In [44]:
```
soup.a
```

Out[44]:
```
<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>
```

- is there any way that help us to extract the attribute values?

In [45]:
```
soup.a['class']
```

Out[45]:
```
['sister']
```

In [46]:
```
soup.a.get('class')
```

Out[46]: ['sister']

In [47]:
```
soup.a['href']
```

Out[47]: 'http://example.com/elsie'

In [48]:
```
soup.a.get('href')
```

Out[48]: 'http://example.com/elsie'

In [49]:
```
soup.a['id']
```

Out[49]: 'link1'

In [50]:
```
soup.a.get('id')
```

Out[50]: 'link1'

In [ ]: